

SIMATS ENGINEERING

CO1-ASSESSMENT TOOL-3

COURSE NAME: Theory of Computation COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO1: Analyze fundamental mathematical concepts of automata theory for formal languages and decision problems(BL:3)

Assessment Tool Weightage: 10%

QUESTIONS: 4 Total Marks:20 Duration : 1 Hour

Technical Presentation

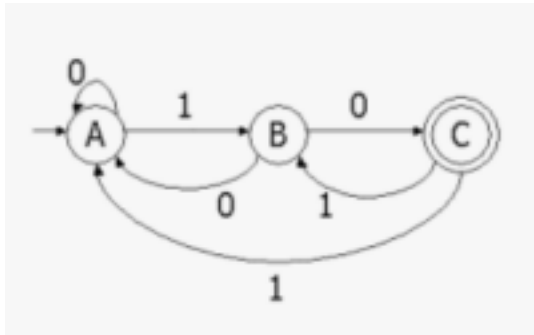
Code of Conduct:

I GIRIJA B / 192311044 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: GIRIJA B

1. How can every Non-Deterministic Finite Automaton (NFA) be converted into an equivalent Deterministic Finite Automaton (DFA), and what are the challenges involved? Consider the NFA given below which accepts some language L over the alphabet $\Sigma = \{0,1\}$. Construct a DFA that represents the same language L



Challenges in NFA → DFA Conversion

- An NFA can have multiple transitions for the same input.
- An NFA may have no transition for an input.
- A DFA must have exactly one transition for every input from each state.
- The number of DFA states may grow up to 2^n , where n is the number of NFA states.

Step 1: NFA Description

From the given diagram:

States: {A, B, C}

Alphabet: {0,1}

Start State: A

Final State: C

Transition Table of NFA

State	Input 0	Input 1
A	{A}	{B}
B	{A,C}	$\emptyset$
C	$\emptyset$	{A,B}

Step 2: Subset Construction

Start state of DFA = {A}

DFA State Transitions

DFA State	Input 0	Input 1
→ {A}	{A}	{B}
{B}	{A,C}	$\emptyset$
*{A,C}	{A}	{A,B}
{A,B}	{A,C}	{B}
$\emptyset$	$\emptyset$	$\emptyset$

Here,

- $\emptyset$ is the dead state.
- Any DFA state containing C is an accepting state.

Step 3: Equivalent DFA

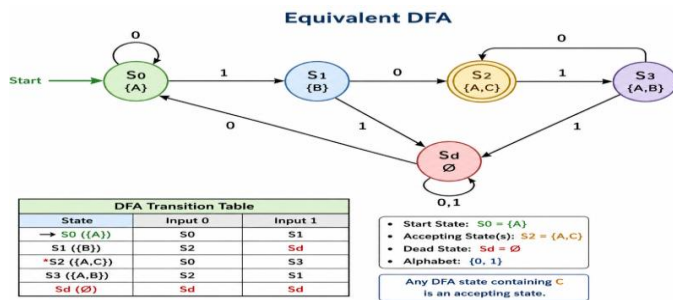
DFA States

- $S_0 = \{A\}$ (Start)
- $S_1 = \{B\}$
- $S_2 = \{A,C\}$ (Final)
- $S_3 = \{A,B\}$
- $S_d = \emptyset$ (Dead)

DFA Transition Table

State	0	1
→ S0 ({A})	S0	S1
S1 ({B})	S2	Sd
*S2 ({A,C})	S0	S3
S3 ({A,B})	S2	S1
Sd (∅)	Sd	Sd

DFA State Diagram



2. How is mathematical induction used to prove properties of strings and languages in automata theory? Apply the principle of mathematical induction and prove that for all positive integers n , the statement given below is true.

$$\sum_{i=1}^n i \cdot i! = (n+1)! - 1$$

(a) Role of Mathematical Induction in Automata Theory

Mathematical induction is a proof technique used to establish that a statement is true for all positive integers. In automata theory, it is commonly used to prove properties of:

- Strings of length n
- Languages generated by grammars
- Recursive definitions
- Correctness of automata and algorithms
- Closure properties of regular languages

The induction process consists of:

1. Base Case: Verify the statement for the smallest value (usually $n = 1$).
2. Induction Hypothesis: Assume the statement is true for $n = k$.
3. Induction Step: Prove it is true for $n = k + 1$.

(b) Prove by Mathematical Induction

Prove that $\sum_{i=1}^n i^2 = \frac{n(n+1)(2n+1)}{6}$
for all positive integers n .

Step 1: Base Case ($n = 1$)

$$\text{LHS: } \sum_{i=1}^1 i^2 = 1^2 = 1$$

$$\text{RHS: } \frac{1(1+1)(2(1)+1)}{6} = \frac{1 \times 2 \times 3}{6} = \frac{6}{6} = 1$$

Hence, LHS = RHS

The statement is true for $n = 1$.

Step 2: Induction Hypothesis

Assume the statement is true for $n = k$.

$$\text{That is, } \sum_{i=1}^k i^2 = \frac{k(k+1)(2k+1)}{6}$$

Step 3: Induction Step

We must prove the formula for $n = k + 1$.

$$\text{LHS: } \sum_{i=1}^{k+1} i^2 = \sum_{i=1}^k i^2 + (k+1)^2$$

$$\text{Using the induction hypothesis, } \frac{k(k+1)(2k+1)}{6} + (k+1)^2$$

$$\text{Take the common factor } (k+1): = (k+1) \left[\frac{k(2k+1)}{6} + (k+1) \right]$$

$$\text{Take LCM} = 6: = (k+1) \left[\frac{k(2k+1)+6(k+1)}{6} \right]$$

$$\text{Expand:} = (k+1) \left[\frac{2k^2+k+6k+6}{6} \right] = (k+1) \left[\frac{2k^2+7k+6}{6} \right]$$

$$\text{Factorize: } 2k^2 + 7k + 6 = (2k+3)(k+2)$$

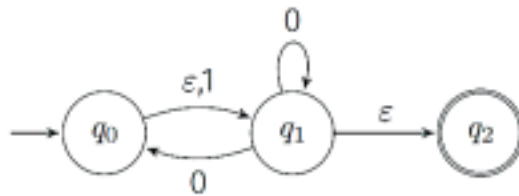
$$\text{Therefore, } = \frac{(k+1)(k+2)(2k+3)}{6}$$

$$\text{Since, } 2(k+1) + 1 = 2k+3,$$

$$\text{we obtain } = \frac{(k+1)((k+1)+1)(2(k+1)+1)}{6}.$$

This is exactly the required formula for $n = k + 1$.

3. What role do epsilon (ϵ) transitions play in finite automata, and how can ϵ -NFA be converted into an equivalent NFA without epsilon transitions? construct an equivalent NFA without ϵ -transitions that accepts the same language. Draw its transition table and mark the initial and final states



Role of ϵ -Transitions

An **ϵ -transition** allows an automaton to move from one state to another **without consuming any input symbol**.

Uses of ϵ -transitions:

- Simplify the construction of NFAs.
- Combine smaller automata into larger ones.
- Represent optional moves and branching.
- Make regular expression to NFA conversion easier.

To convert an **ϵ -NFA to an equivalent NFA**, follow these steps:

1. Compute the **ϵ -closure** of every state.
2. For each input symbol, find transitions using the ϵ -closures.
3. Mark a state as **final** if its ϵ -closure contains a final state.
4. Remove all ϵ -transitions.

Given ϵ -NFA

States

- $Q = \{q_0, q_1, q_2\}$

Alphabet

- $\Sigma = \{0, 1\}$

Start State

- q_0

Final State

- q_2

Transitions

- $q_0 \xrightarrow{\epsilon} q_1$
- $q_0 \xrightarrow{1} q_1$
- $q_1 \xrightarrow{0} q_1$
- $q_1 \xrightarrow{0} q_0$
- $q_1 \xrightarrow{\epsilon} q_2$

Step 1: ϵ -Closures

State	ϵ -Closure
q_0	$\{q_0, q_1, q_2\}$
q_1	$\{q_1, q_2\}$
q_2	$\{q_2\}$

Step 2: Equivalent NFA (Without ϵ)

Using

$$\delta'(q, a) = \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(q)a))$$

Transition Table

State	Input 0	Input 1
$\rightarrow^* q_0$	$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$

State	Input 0	Input 1
*q ₁	{q ₀ , q ₁ , q ₂ }	∅
*q ₂	∅	∅

Initial State

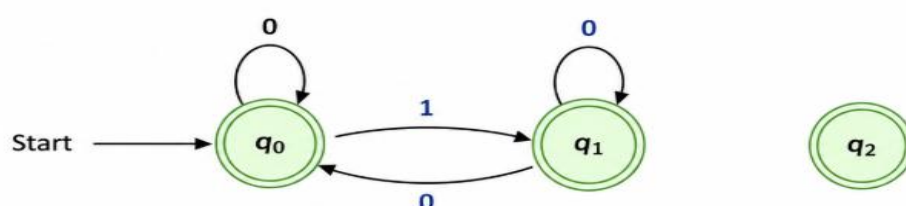
- q₀

Final States

Since the ϵ -closures of q₀ and q₁ contain q₂, the final states are:

- q₀
- q₁
- q₂

Equivalent NFA (Without ϵ)



Final States:

((q₀)), ((q₁)), ((q₂))

Transitions:

q₀ --0--> q₀, q₁, q₂

q₀ --1--> q₁, q₂

q₁ --0--> q₀, q₁, q₂

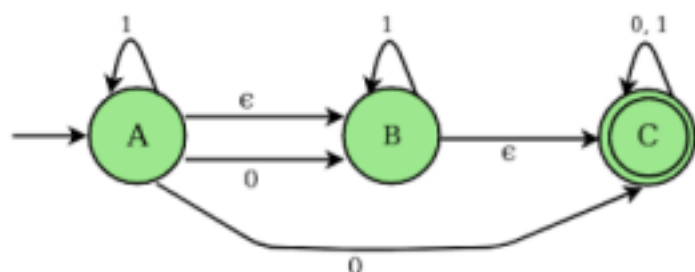
q₂ has no outgoing transitions

Final Answer

- **Initial State:** → q₀
- **Final States:** q₀, q₁, q₂
- **Equivalent NFA Transition Table:**

State	0	1
→ *q ₀	{q ₀ , q ₁ , q ₂ }	{q ₁ , q ₂ }
*q ₁	{q ₀ , q ₁ , q ₂ }	∅
*q ₂	∅	∅

4.A NFA with ϵ -transitions is given below. Find ϵ -closure for each state and construct an equivalent DFA which accepts the same language. Draw its transition table and mark the initial and final states



Step 1: ε-Closure of Each State

From the given ε-NFA:

- **States:** {A, B, C}
- **Alphabet:** {0,1}
- **Start State:** A
- **Final State:** C

Transitions:

- A --ε→ B
- A --0→ B
- A --0→ C
- A --1→ A
- B --1→ B
- B --ε→ C
- C --0→ C
- C --1→ C

ε-Closures

State	ε-Closure
A	{A, B, C}
B	{B, C}
C	{C}

Step 2: Equivalent DFA (Subset Construction)

DFA States

DFA State	NFA States
S0	{A,B,C} (Start, Final)
S1	{B,C} (Final)
S2	{C} (Final)

DFA Transition Table

DFA State	Input 0	Input 1
→ *S0 = {A,B,C}	S0	S0
*S1 = {B,C}	S2	S1
*S2 = {C}	S2	S2

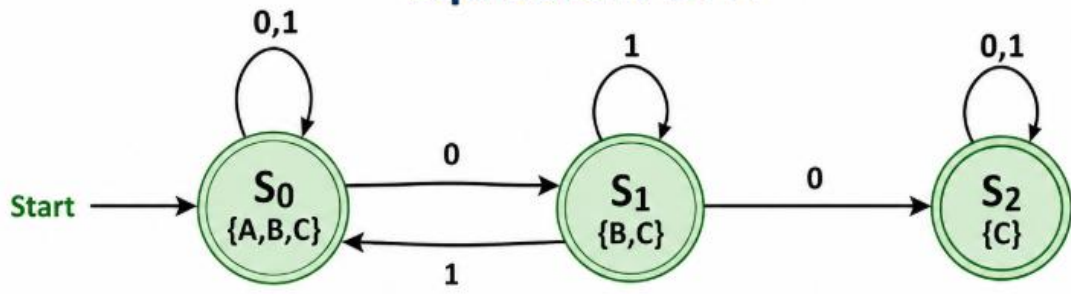
Initial State: S0 = {A,B,C}

Final States: S0, S1, S2 (all contain C)

Transition Diagram Summary

- **S0 = {A,B,C}**
 - 0 → S0
 - 1 → S0
- **S1 = {B,C}**
 - 0 → S2
 - 1 → S1
- **S2 = {C}**
 - 0 → S2
 - 1 → S2

Equivalent DFA



W